Resumen

Este manual describe la arquitectura, componentes y operacion tecnica de SOS EmergeLens: frontend React/Vite, backend Flask, integracion con Odoo, voz (SpeechRecognition/TTS), IA (Groq) y reportes (ReportLab).

Stack tecnologico

- Frontend: React (Vite), CSS custom, Leaflet para mapas.
- Backend: Python + Flask, sesiones por cookies, CORS con credenciales.
- ERP/DB: Odoo 17 sobre Postgres 15 (modelos x.emergelens.*).
- IA: Groq Chat Completions (LLM) y Whisper para transcripcion de audio.
- Reportes: ReportLab (generadores en reportland/).

Arquitectura (alto nivel)

La aplicacion corre en 4 servicios principales (docker-compose): db (Postgres), odoo, backend y frontend. El frontend consume el backend via /api. El backend usa JSON-RPC/HTTP para leer/escribir en Odoo.

Frontend (5173)	**->**	**Backend Flask (5000)**	**->**	**Odoo (8069)**	**->**	**Postgres (5432)**

Estructura del repositorio

frontend/ UI React (pages/, components/, hooks/, lib/).
backend/ API Flask (routes/, scheduler.py, security.py, validation.py).
odoo/ Addons custom (modelos y vistas en Odoo).
reportland/ Generadores de PDF/HTML (ReportLab).

Flujo de sesion y seguridad

• El backend configura cookies de sesion HttpOnly y SameSite (configurable por env).
• CORS soporta credenciales; origins se controla con CORS_ORIGINS.
• Auditoria: before/after_request registra acciones criticas (perfil, contactos, evidencia, status).

API (resumen)

Grupo	Ejemplos de endpoints
Auth	/api/auth/login, /api/auth/register, /api/auth/me
Perfil	/api/profile/, /api/profile/instructions, /api/profile/by-emergelens-id/<id>
Contactos	/api/contacts/ (CRUD)
Emergencia	/api/emergency/email, /api/emergency/my-alert, /api/emergency/evidence, /api/emergency/stop
Chat IA	/api/chat/... (conversaciones, mensajes, audio)
Llamada LENS	/api/lens/message
Geofence	/api/geofence/...
Donaciones	/api/donations/...
Reportes	/api/reports/...

Voz (CallSimulator)

La llamada usa Web Speech API del navegador: SpeechSynthesis (TTS) para hablar y SpeechRecognition para capturar voz. Para evitar cortes o loops, se usa reconocimiento continuo y 'commit por silencio'.
• Frontend: frontend/src/pages/CallSimulator.jsx.
• Backend: /api/lens/message en backend/routes/lens_call.py.

IA (Groq) y anti-repeticion

El backend arma un system prompt con reglas estrictas (max 2 oraciones, una pregunta) y pasa historial corto. Adicionalmente aplica filtros server-side para bloquear frases repetidas o preguntas ya hechas y reemplaza por un fallback contextual.
• Archivo: backend/routes/lens_call.py.

Despliegue local (Docker Compose)

Comandos tipicos:

```
docker compose up --build
```

Puertos:

• Frontend: http://localhost:5173
• Backend: http://localhost:5000/api/health
• Odoo: http://localhost:8069

Variables de entorno (principales)

• SECRET_KEY, SESSION_DAYS, SESSION_COOKIE_SECURE, SESSION_SAMESITE
• CORS_ORIGINS
• ODOO_URL, ODOO_DB, ADMIN_ODOO_EMAIL, ADMIN_ODOO_PASS
• GROQ_API_KEY, GROQ_MODEL

Reportes PDF (ReportLand)

Los documentos imprimibles se generan por codigo en reportland/. Ejemplo: cronograma_reportlab.py genera cronograma_proyecto.pdf.
• Generar este manual: python reportland/manual_tecnico_reportlab.py --out reportland/manual_tecnico.pdf
• Generar manual de usuario: python reportland/manual_usuario_reportlab.py --out reportland/manual_usuario.pdf